Calibration Objectives and Evaluation Protocols in 4-bit Edge LLM Deployment

A controlled measurement study on Rockchip RK3576 — draft v0.1

NAC project · internal draft for review

2026-10-30

Abstract

Weight-only 4-bit quantization is the default way to deploy LLMs on edge NPUs, and its calibration is almost always evaluated in a floating-point simulation. We present a controlled study of five calibration objectives (min-max, GPTQ, AWQ, NAC-v1, NAC-v2) under one W4A16 pipeline, one 1.5B model and one edge SoC (Rockchip RK3576). Four findings emerge. First, objectives that are indistinguishable on perplexity and multiple-choice accuracy differ by more than 4× on chain-of-thought accuracy: GPTQ reaches 0.305 on GSM8K while min-max and NAC-v1 reach 0.075 and 0.070, although NAC-v1 has the best perplexity of the three. Second, much of that gap is measurement: it falls to about 1.3× when scoring anchors on the answer sentence, because the activation-weighted methods continue into a new question after answering, which last-number extraction scores as an arithmetic error. Third, the protocol itself can erase the result: a 32-token generation cap drives every configuration, including the vendor reference model, to 0.000–0.025, while the same local artifacts score 0.535–0.565 at 512 tokens — all statistically indistinguishable from the vendor ($p = 0.69$–1.00). Fourth, calibration effects are bounded by group size — NAC-v1 loses 7.43 points of MMLU to min-max at g64 and gains 6.71 at g128 — and, on the deployment stack, a local rebuild with the toolkit's default algorithm loses 6.1 points to the vendor artifact ($p = 3e\text{-}4$) until a different build algorithm is used ($p = 0.18$). We give a reproducible protocol, harness and checklist so that these confounds are visible. We make no state-of-the-art claim.

Keywords: post-training quantization; edge NPU; W4A16; evaluation protocol; chain-of-thought; calibration

1 Introduction

Deploying a 1.5B-parameter language model on a 16 GB edge SoC is routine: quantize the weights to 4 bits, calibrate on a small corpus, export to the vendor runtime. The calibration step is where most published methods compete, and it is evaluated almost universally in simulation — apply the quantizer's `dequant(quantize(W))` to each weight, run the model in fp32, and report perplexity and zero-shot accuracy. The implicit assumption is that simulation is a faithful proxy for deployment.

This paper asks what that assumption costs. We do not propose a new quantizer; we hold the deployment stack fixed and vary the calibration objective, then measure what survives when the metric moves from next-token likelihood to multi-step generation.

Our study is organised around three axes that are usually conflated (Section 3): the quantizer (toolkit-native min-max/GPTQ versus externally calibrated NAC/AWQ), the

execution stack (x86 fp32 pseudo-quantization versus the RK3576 NPU), and the evaluation protocol (task, shots, generation cap, answer extraction). Every table in this paper labels all three.

Finding 1 (calibration objective). In simulation, GPTQ's Hessian-based error compensation is worth 23 points of GSM8K accuracy over min-max and NAC-v1 (0.305 versus 0.075 and 0.070), while NAC-v1 has the best perplexity of the three (11.893 versus 12.168 and 12.225). Perplexity and long-form accuracy rank the methods in opposite directions. The 23-point gap is not a pure arithmetic effect: under an answer-anchored extractor it shrinks to about 8 points, and what remains tracks a behavioural difference — how often the model continues into a new question after answering (11% for GPTQ versus 30–53% for the activation-weighted methods).

Finding 2 (protocol). The same measurements are extremely sensitive to how generation is scored. With a 32-token cap — the default of the evaluation harness and of several published edge-deployment recipes — the vendor's own W4A16 model scores 0.0000 on GSM8K, and ten locally built variants score 0.000–0.025. Extending the cap to 512 tokens restores the vendor model to 0.5550, and restores one of those local artifacts from 0.0150 to 0.5350 (statistically tied with the vendor, p = 0.69): the "long-form failure" was the cap. Separately, extracting the first "answer is" number instead of the last number in the generation raises every configuration and compresses the GPTQ advantage from 4.1× to about 1.3×. Neither artefact is visible in the accuracy scalar alone; both are visible in the length distribution and in the termination rate.

Finding 3 (boundary). Calibration effects are conditional on group size. At g64, NAC-v1 loses 7.43 points of MMLU to min-max (p = 3.3e-9); at g128 it wins by 6.71 points (p = 7.9e-9). The sign of the effect, not just its size, depends on a hyperparameter that papers rarely ablate.

Finding 4 (build path). On the NPU, where single-letter MMLU is insensitive to the cap, a local rebuild of the same model with the toolkit's default algorithm scores 0.4974 against the vendor artifact's 0.5583 (−6.09 pp, p = 3.0e-4). Switching the build algorithm to `gdq` closes the gap to a statistical tie (0.5387, p = 0.18). The vendor's algorithm is not documented, so "rebuild and compare" is not a controlled experiment unless the algorithm is fixed.

Contributions. 1. A three-axis evaluation protocol with a reproducible harness that archives full generations, token counts, and termination flags (Section 3, Appendix B). 2. A controlled comparison of five calibration objectives under one W4A16 pipeline, showing that likelihood and multiple-choice metrics invert the long-form ranking (Section 6.1). 3. A quantified account of two protocol artefacts — the generation cap and the extraction rule — that are large enough to reverse published conclusions (Section 6.2), with the vendor model as the control. 4. A boundary characterisation of calibration effects along group size and task type (Section 6.3). 5. A build-reproducibility result: the same model rebuilt with the toolkit's default algorithm loses 6.1 points of MMLU to the vendor artifact, and only a different quantization algorithm closes the gap (Section 6.5). 6. A protocol checklist for edge quantization papers, in which every item corresponds to a measured failure in this study rather than to intuition (Section 7.1).

We do **not** claim state of the art, that one calibration family dominates, or that any of these effects transfer to other SoCs without replication.

---

2 Related Work

Post-training quantization. GPTQ [Frantar et al., 2023] minimises layer-wise reconstruction error with a Hessian-based update in column order. AWQ [Lin et al., 2024] searches per-channel scales that protect salient activation channels and folds them into adjacent weights. SmoothQuant [Xiao et al., 2023] migrates activation outliers into weights, and ZeroQuant [Yao et al., 2022] combines group-wise weight quantization with layer-wise distillation. OmniQuant [Shao et al., 2024] learns clipping and transformation parameters, and rotation-based methods such as QuaRot [Ashkboos et al., 2024] remove outliers before quantization. All of these optimise a simulated objective, and all are evaluated primarily by perplexity plus zero-shot accuracy on classification-style tasks; the scaling behaviour of k-bit inference is itself the subject of [Dettmers & Zettlemoyer, 2023].

Evaluation validity. The observation that accuracy is insufficient to characterise compressed models is not new — [Accuracy is Not All You Need, NeurIPS 2024] makes the general case for compression. What is missing is a deployment-side account: which protocol choices move the number, by how much, and in which direction. Our §6.2 is an attempt at that accounting on a concrete NPU.

Edge NPU deployment. Vendor toolchains (rknn-llm, MNN, ExecuTorch, TFLite) expose integer-domain quantization parameters and their runtimes implement integer dequantization. Public documentation of the exact asymmetry semantics is thin, and recent work on Ascend NPUs [arXiv:2602.17693] reports that PTQ baselines behave differently on-device than in simulation. Our study is complementary: it fixes one SoC and one model and asks how much of the simulation-deployment gap is attributable to calibration versus measurement.

Positioning. This is a measurement paper. Its unit of contribution is a reproducible discrimination — a protocol under which two calibration objectives can be told apart — plus the boundary conditions under which that discrimination holds.

---

3 Evaluation Framework

3.1 Three axes

Axis	Levels	Why it must be labelled
Quantizer	toolkit-native (min-max, GPTQ) / externally calibrated (NAC-v1, NAC-v2, AWQ)	The calibration corpus and the numerical path differ
Stack	x86 fp32 pseudo-quantization / RK3576 NPU	Absolute numbers are not comparable across stacks
Protocol	task × shots × generation cap × extraction rule	Changes the ranking (Section 6.2)

Rule. No table mixes levels of the stack axis; no claim crosses the stack axis without an explicit “not comparable in absolute terms” caveat.

3.2 Deployment-level protocol

Hardware: NanoPi M5 (RK3576), 4×Cortex-A53 @2.016 GHz + 4×Cortex-A72 @2.208 GHz, 16 GB LPDDR5, 2-core NPU @950 MHz, fixed-frequency operation (CPU/NPU/DDR) for all timed runs. Runtime: librkllm 1.2.3.

Models: Qwen2.5-1.5B-Instruct (full matrix) and Qwen2.5-3B-Instruct (vendor-quantized reference point only).

Tasks: MMLU [Hendrycks et al., 2021] (969 items, zero-shot, 32-token cap, single-letter answer), GSM8K [Cobbe et al., 2021] (200 items, zero-shot for deployment and 2-shot CoT for simulation, 512-token cap), WikiText-2 [Merity et al., 2017] held-out perplexity (64 sequences × 1024 tokens).

3.3 What we archive

For every item we store the full generation (up to 4000 characters), the token count, whether the cap was hit, and both extraction results (last number and first “answer is”). This is a direct response to the failure mode described in §6.2: an accuracy scalar computed from an unarchived generation cannot be audited, and cannot be re-scored under a different rule.

---

4 Calibration Objectives

All methods produce per-group `(scale, zero_point)` for the deployment kernel’s dequantization `W' = scale_g · (q − zp_g)`, `q ∈ [0,15]`, group size 64 or 128.

Min-max (toolkit `normal`). Per-group extremes; no error compensation. Represents the “no calibration objective” baseline.

GPTQ. Hessian-based column-order compensation with `percdamp = 0.01` and a max-shrink grid (0.8, 100 points); integer zero-points; 6527 s calibration on the board’s CPU.

AWQ (re-implementation). Activation-aware per-channel scale search (α grid, 21 points, step 0.05), clip grid, exact folding into adjacent weights, integer zero-points; 1511 s. Fold correctness is asserted separately: a fold-only pass reproduces fp16 perplexity exactly (10.627), which bounds implementation error before any accuracy claim.

NAC-v1. GPTQ-style sequential block propagation plus an activation-weighted per-group truncation-ratio grid (κ ∈ {1.0, 0.95, …, 0.4}) minimising `Σ_g a ⊙ (W − dequant(q(W; κ))²`; floating-point zero-points; 521 s — the cheapest method in the study.

NAC-v2. NAC-v1 plus integer zero-points and block-wise Hessian compensation (`act_weight = diag(H)`, `percdamp = 0.01`); 4259 s.

On attribution. NAC-v2 changes two things at once (zero-point rounding and Hessian compensation). In simulation, integer zero-points change only rounding, so the simulated gain is attributable to the Hessian term; this is a prediction, not a measured decomposition, and we do not report a per-component split.

---

5 Experimental Setup

Calibration. All objectives use the same calibration corpus: 128 WikiText-2 sequences of 512 tokens, of which 8 sequences (4096 tokens) are used for activation statistics and

Hessian accumulation; seed 42; float32 on CPU. NAC's κ grid is {1.0, 0.95, 0.9, 0.85, 0.8, 0.7, 0.6, 0.5, 0.4}, searched per group. GPTQ and NAC-v2 use `percdamp = 0.01` and the same column ordering.

Simulation. `apply_qparams` replaces each linear weight by `dequant(quantize(W))` layer by layer with float32 arithmetic, using the same `scale·(q − zp)` formula as the kernel and the same group size. No activation quantization is applied (W4A16). Generation is greedy (`top_k = 1`), with `keep_history = 0` so items are independent.

Deployment. RKLLM runtime 1.2.3, context 4096, embedding cache enabled (`embed_flash = 1`), 4×A72 cores enabled. All timed runs use fixed frequencies (CPU 2016/2208 MHz, NPU 950 MHz, DDR 2736 MHz) verified before each run. Model dtype and artifact md5 are recorded for every run.

Perplexity. WikiText-2 held-out, 64 sequences × 1024 tokens for all configurations except AWQ's probe (16 sequences), labelled wherever it appears.

Statistics. McNemar's test with continuity correction for paired binary outcomes; bootstrap 95% confidence intervals over items; tie criterion $p > 0.05$ together with a difference smaller than the CI width. Pairing is by item index, verified by asserting that the gold-answer sequence is identical between the two runs. Full pairwise statistics, including odds ratios, are in Appendix C. `analysis/check_claims.py` re-derives every number in this paper from the archived JSON and fails if any of them drifts.

---

6 Results

6.1 Calibration objective: likelihood and multiple-choice invert long-form ranking

Table 1. Simulated calibration objectives, one model (Qwen2.5-1.5B), one dtype (W4A16), group size 64.

Method	MMLU (969)	GSM8K (200)	PPL (64 seq)	Calib. time
fp16	0.4634	[[TBD: fp16 anchor]]	11.075	—
min-max	0.4675	0.075	12.225	—
GPTQ	0.4520	**0.305**	12.168	6527 s
AWQ	0.4396	0.000 (n=64)	11.226[1]	1511 s
NAC-v1	0.3932	0.070	**11.893**	521 s
NAC-v2	0.4469	0.150	11.962	4259 s

[1] 16-sequence probe; all other perplexities are 64-sequence.

Two orderings coexist in this table. By perplexity, NAC-v1 is best among the quantized methods and GPTQ is worst; by GSM8K, GPTQ is 4.1× better than NAC-v1 and 4.3× better than min-max (Figure 1). MMLU does not discriminate: NAC-v1's 7.43-point simulated deficit to min-max ($p = 3.3e\text{-}9$) is real, but it is the opposite of what the long-form task shows, and the deployment-side MMLU comparison (Section 6.5) shows only a build-path effect, not a calibration effect.

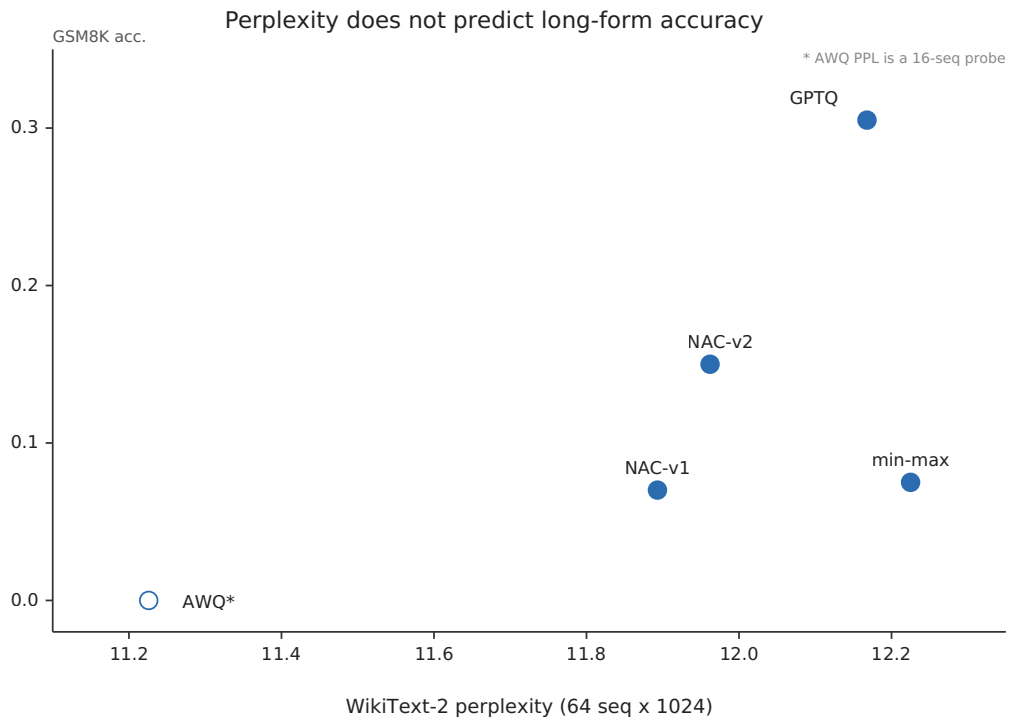


Figure 1. Perplexity versus GSM8K accuracy for the five calibration objectives. The two axes rank the methods in opposite directions; NAC-v1 has the best perplexity and the worst long-form accuracy among the quantized variants. AWQ's perplexity is a 16-sequence probe (open marker).

The 4× is partly an extraction effect, and we say so. Re-scoring archived generations with an answer-anchored rule ("first `answer is` number") instead of the last number raises every configuration and compresses the gap to about 1.3×:

Method	last-number (stored)	answer-is (prefix recompute)
GPTQ	0.305	0.385
NAC-v2	0.150	0.340
AWQ	0.000	0.359
min-max	0.075	0.310
NAC-v1	0.070	0.300

The prefix recomputation is a lower bound (archives truncate at 300 characters), so these numbers are indicative, not final; the re-run with full archives will replace them. What survives is a behavioural difference rather than an arithmetic one: within the first 300 characters, GPTQ continues into a new question in 11% of items, versus 30–53% for the activation-weighted methods (min-max 38%, NAC-v1 30%, NAC-v2 37%, AWQ 53%). Under a last-number metric, continuation into the next question is scored as an arithmetic error. The honest statement is therefore: **calibration objectives differ in whether the model stops after answering, and a last-number metric converts that difference into an apparent 4× accuracy gap.**

The pattern is not a single-method accident. AWQ — an independent activation-weighted family with a different parameterisation — matches the same signature: best-in-class perplexity among the methods we could measure at 16 sequences (11.226), MMLU statistically tied with GPTQ and NAC-v2 ($p = 0.369$ and 0.579), and a GSM8K collapse that is not an accuracy failure but a termination failure: generations remain coherent and

often contain the correct final number, but the model does not emit EOS and continues into the next question, so last-number extraction reads a number from the continuation. Under answer-is extraction the same 64 items score 0.359 instead of 0.047.

Interpretation. Likelihood averages over token positions and is dominated by high-probability continuations; it is insensitive to the low-probability tail where EOS lives and where multi-step arithmetic errors accumulate. A calibration objective that minimises reconstruction error can therefore improve perplexity while degrading the termination policy.

6.2 Protocol artefacts

6.2.1 The generation cap

The strongest evidence that the cap, not the model, can determine the score comes from a within-model manipulation. We evaluated the vendor-provided W4A16 model — the reference artifact, untouched by any of our calibration code — on the same 200 GSM8K items, changing only `max_new_tokens`:

Model	max_new_tokens	GSM8K	median generation length
vendor W4A16	32	**0.0000**	140 chars
vendor W4A16	512	**0.5550**	>200 chars (archive capped)

At 32 tokens the reference model scores zero. Its generations are not degenerate: they begin with a correct restatement of the problem and a correct first step, and stop mid-number — for the first item, `"…divide by 3.\n\nThe first measurement is 47 W"`. The harness's last-number extractor reads `47` and marks the item wrong. A second vendor model (Qwen2.5-3B, same harness, 512 tokens) scores 0.6700, so the harness is not degenerate by construction: it is the cap that is.

The same manipulation applies to a locally rebuilt artifact, and it is the clearest single result in this paper:

Artifact	GSM8K @32 tok	GSM8K @512 tok
vendor W4A16	0.0000	0.5550
local rebuild (`normal`, opt=1, W4A16)	0.0150	**0.5350**
local group-quantized build (`W4A16_G64`, the artifact an earlier version of this work labelled a calibration collapse)	0.0050	**0.5650**
second group-quantized build (alternate calibration, `W4A16_G64`)	0.0050	**0.5600**

Nothing about these models changes between the two columns except the cap. At 512 tokens all three are statistically indistinguishable from the vendor artifact (paired McNemar: p = 0.69, 0.88 and 1.00 respectively), and the 52–56-point differences within each row are artefacts of the harness. Any conclusion of the form "the local build loses long-form reasoning" drawn from a 32-token run is wrong.

The 512-token protocol is also clean on its own terms: across the three artifacts re-measured with the instrumented harness, the cap was hit in only 1.5–2.0% of items (median generation 235–248 tokens), and no generation opened a new question after its

answer (cross-question rate 0.00). Under 32 tokens, the same artifacts hit the cap in essentially every item and never reach an answer sentence (Table 2).

For the third row the point is sharper. That artifact was, in an earlier version of this work, the central evidence for a hardware-semantics hypothesis: it scored 0.0050 at 32 tokens and was read as numerical corruption. At 512 tokens it scores 0.5650 — above the vendor's 0.5550, with identical last-number and answer-anchored scores (0.5650/0.5650), i.e. no continuation contamination at all. A second, independently calibrated artifact behaves the same way (0.0050 → 0.5600). The "collapse" was the cap.

The same signature appears in every locally built variant we evaluated at 32 tokens (Table 2, Figure 2): median length 129–147 characters, 90th percentile ≤161, no generation reaches a concluding sentence, and the distribution of final characters is dominated by digits and operators. Under a 512-token cap the vendor model reaches 0.5550. We therefore treat all 32-token numbers in our own earlier records — and any published edge-deployment number produced with a similar default — as measuring the cap.

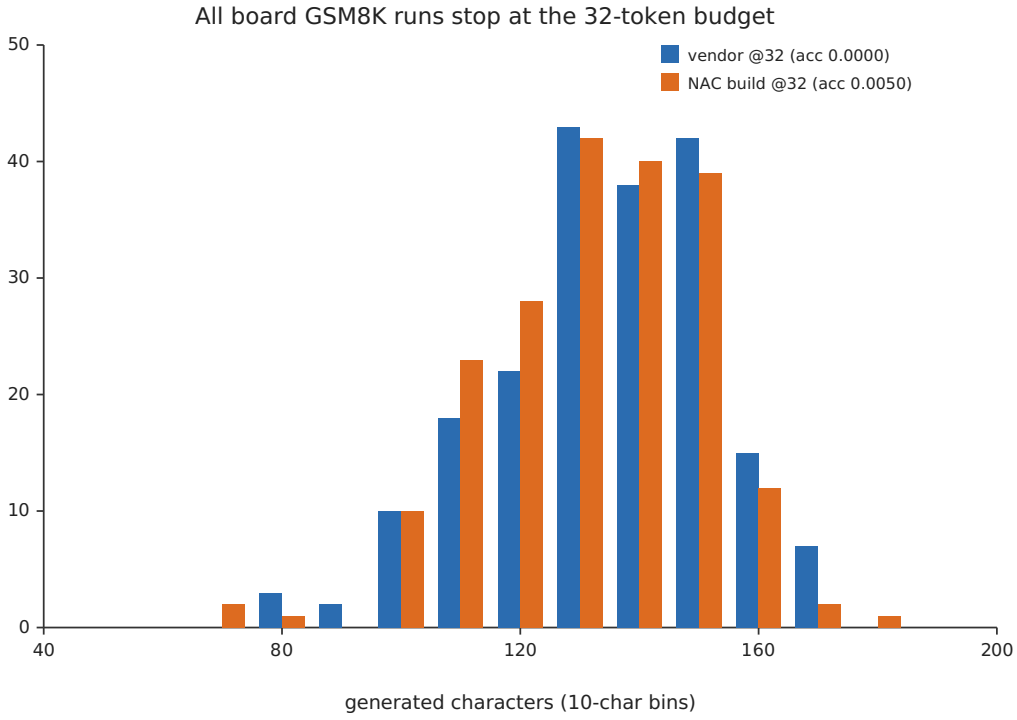


Figure 2. Length distribution of GSM8K generations under a 32-token cap, vendor model versus one locally built variant. Both distributions terminate against the same budget; neither reaches a concluding answer sentence.

Table 2. Truncation signature of GSM8K runs. `tail-mid` is the fraction of generations whose last character is a digit or operator; `hasAns` is the fraction containing a concluding "answer is".

Run (all 32-token unless noted)	n	GSM8K	p50 len	p90 len	tail-mid	hasAns
vendor	200	0.0000	140	160	0.32	0.00
min-max (toolkit)	200	0.0050	147	161	0.32	0.00
NAC-v1 calibration artifact	200	0.0050	138	158	0.28	0.00
NAC-v2 calibration artifact	200	0.0050	138	158	0.28	0.00

Run (all 32-token unless noted)	n	GSM8K	p50 len	p90 len	tail-mid	hasAns
local x86 build, alg=normal	200	0.0150	138	157	0.38	0.00
local x86 build, alg=gdq	200	0.0100	145	161	0.28	0.00
vendor, 512-token	200	**0.5550**	200*	200*	0.23	0.00[1]

* archive capped at 200 characters; the scoring used full text. [1] “answer is” appears only after the first 200 characters, which is why archived prefixes cannot be re-scored (Section 6.2.2).

This is a measurement-validity problem rather than a modelling result, and it has a simple control: **run the reference model under the same harness**. If the reference scores near zero, the harness is the treatment.

6.2.2 The extraction rule

Re-scoring archived generations with the “answer is” rule raises every simulated configuration: GPTQ 0.280→0.385, NAC-v2 0.130→0.340, min-max 0.035→0.310, NAC-v1 0.095→0.300, AWQ 0.047→0.359. These are lower bounds: the archives truncate at 300 characters, so the true effect is at least this large. The GPTQ margin over NAC-v1 falls from 4.1× to about 1.3×.

The mechanism is visible in AWQ’s case. Its generations are coherent and often contain the correct final number, but the model does not emit EOS; it continues into a new question, and last-number extraction reads a number from the continuation. Under answer-is extraction the same 64 items score 0.359 instead of 0.047. The underlying phenomenon — a calibration objective that damages the termination policy while preserving local fluency — is real and worth reporting, but it is not an accuracy collapse, and reporting it as one would be wrong.

We therefore report both rules and treat any conclusion that flips between them as protocol-dependent rather than method-dependent. The re-run additionally truncates each generation at the first `\nQuestion:` before scoring (the installed transformers version does not support generation-time stop strings, so the truncation is applied to the decoded text), which removes the continuation component and lets arithmetic quality and termination behaviour be measured separately rather than as a product.

6.3 Group size reverses the sign

Table 3. The same three objectives at group size 128 (MMLU, 969 items; 64-sequence perplexity).

g128	MMLU (969)	PPL (64 seq)
min-max	0.3622	12.544
NAC-v1	**0.4293** (+6.71 pp, p = 7.9e-9)	**12.164**
GPTQ	0.4871	12.503

The NAC-v1 deficit at g64 (−7.43 pp) becomes a gain at g128 (+6.71 pp). The effect is not a monotone function of “finer grouping is better”: g128 is coarser, and the coarser setting favours the truncation-grid objective while the finer setting favours Hessian compensation (Figure 3). Any claim of the form “method X beats method Y” must therefore carry its group size.

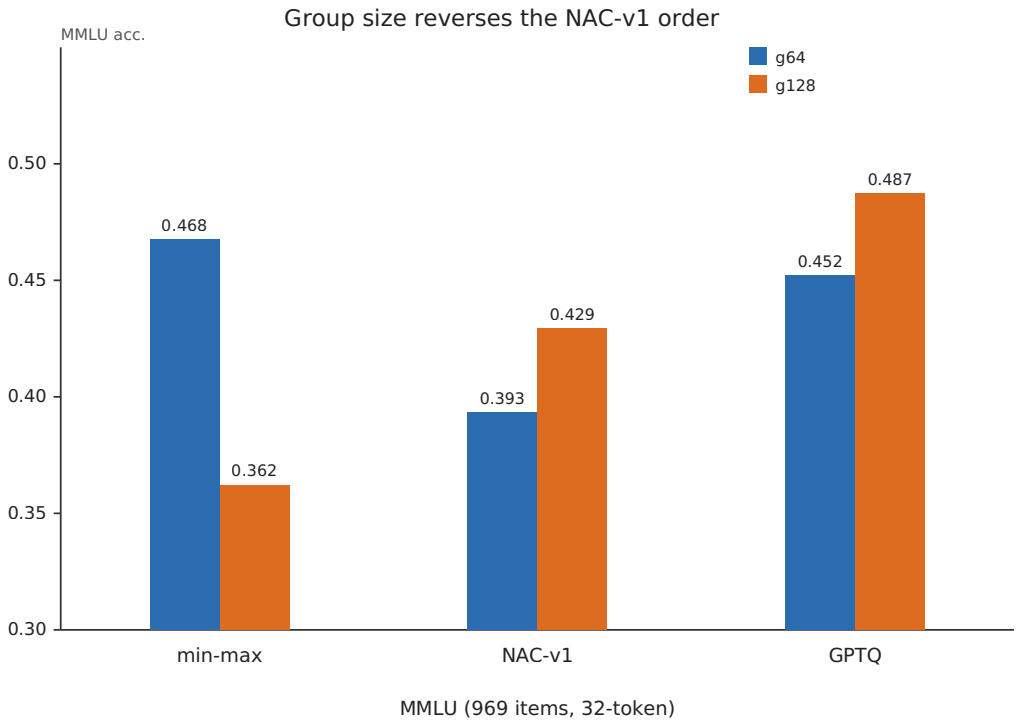


Figure 3. MMLU accuracy at group size 64 versus 128. The ordering of the three calibration objectives is not preserved; NAC-v1 moves from last to second, and min-max moves from first to last.

6.4 Termination behaviour as a first-class metric

A last-number score conflates three failure modes: the model computes the wrong answer, the model does not answer, and the model answers but does not stop. Only the first is a calibration-quality signal. Our archives let us separate the third: among the first 300 characters of each generation, the fraction that opens a new question after the answer is 11% for GPTQ, 30% for NAC-v1, 37% for NAC-v2, 38% for min-max and 53% for AWQ. The rank of these rates is the inverse of the rank of the last-number score, which is what one expects if the metric is reading numbers out of the continuation.

This is not an argument that the continuation behaviour is harmless. In deployment, a model that answers correctly and then continues into a hallucinated next question is a real failure — it wastes decode budget and can corrupt downstream parsing. It is an argument that the failure must be reported as termination, with an EOS rate or a stop-sequence rule, rather than folded into an accuracy number. The re-run (Section 6.2.1) records token counts and cap hits per item, which makes this separation exact rather than prefix-bounded.

6.5 Deployment: the build path is a variable

MMLU is answered with a single letter and is therefore insensitive to the generation cap (Section 6.2), so it can serve as a clean deployment-side probe. Fixing the model, the toolkit version (1.2.3) and the quantization dtype (`W4A16`), we rebuilt locally and varied only the build algorithm:

Table 4. Deployment-side MMLU for the same model and dtype, rebuilt with different toolkit settings.

Build	Algorithm	MMLU (969)	vs vendor	b/c · p
vendor	not disclosed	0.5583	—	—
local, opt=1	`normal`	0.4974	−6.09 pp	158/99 · 3.0e-4
local, opt=3	`normal`	0.5191	−3.92 pp	132/94 · 0.014
local, opt=1, multi-domain calibration	`normal`	0.4974	−6.09 pp	150/91 · 1.9e-4
local, opt=1	`gdq`	0.5387	−1.96 pp	98/79 · 0.18 (tie)

Two readings. First, a local rebuild with the default algorithm is 6.1 points worse than the vendor artifact — significant, and reproduced across two independent builds with different calibration sets (both 0.4974; p = 3.0e-4 and 1.9e-4). Raising the optimization level recovers about a third of the gap (0.5191, p = 0.014). Second, the gap is statistically closed by a different quantization algorithm (`gdq`), which the vendor does not document. “Vendor quality” is therefore not reproducible from the documented default pipeline: the algorithm is a hidden variable, and any study that compares calibration methods on top of a locally rebuilt baseline inherits that uncertainty.

The loss is task-specific, and we report it as such: at 512 tokens the same local build is statistically tied with the vendor on GSM8K (0.5350 vs 0.5550, p = 0.69). The build-path difference therefore shows up on the knowledge-oriented multiple-choice task and not on the arithmetic task, which rules out a generic “the local build is worse” explanation and makes the mechanism an open question (Section 8).

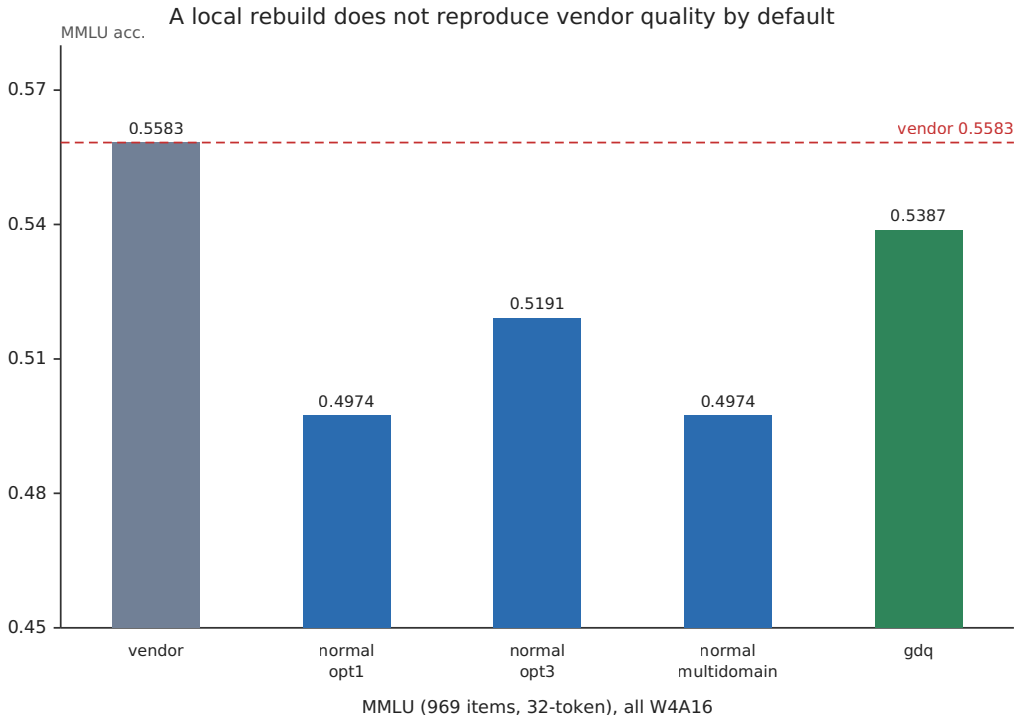


Figure 4. Deployment-side MMLU for the same model and dtype, rebuilt with different toolkit settings. The dashed line is the vendor artifact. Only the `gdq` build recovers it within statistical noise.

The group-quantized artifacts (`W4A16_G64`) are excluded from this table: they differ in dtype, and their apparent advantage over the plain build (+6.40 pp, p=2.8e-5) confounds

granularity with build path. We state this explicitly because an earlier version of this work compared them as if they were the same configuration.

We report deployment-side GSM8K only where the 512-token protocol has completed. Both measured artifacts are statistically indistinguishable from the vendor (0.5550): the plain rebuild at 0.5350 ($p = 0.69$) and the group-quantized build at 0.5650 ($p = 0.88$). We do not report their 32-token numbers, because those measure the cap (Section 6.2.1).

---

7 Discussion

Why likelihood misleads. Perplexity is a mean over positions of log p(token | prefix). Two failure modes are invisible to it: a shift in the EOS decision (a single low-probability event at the end of a long generation) and error accumulation in multi-step arithmetic (errors appear late, after many correct tokens). Both are precisely what edge deployment cares about.

Why the cap matters more than the method. A 32-token cap truncates a chain of thought before its answer in essentially every GSM8K item. The resulting score measures the cap, not the model. Because the cap is a harness default, this failure mode is likely to be widespread; the control that exposes it is the vendor model, which must score well under the same harness if the harness is valid.

Implications for toolchains. If a runtime's dequantization semantics differ from the calibration-side semantics, the difference is invisible in simulation and appears as a deployment-only degradation. Our data cannot yet distinguish such a semantic gap from build-path effects; we report the controlled comparison we can defend and mark the rest as open (Section 8).

7.1 A protocol checklist

Each item below is a mistake we made, found, and fixed. They are cheap to check and each one is capable of reversing a conclusion.

#	Requirement	Failure it prevents	Our case
1	Report the generation cap; set it from the task, not from the harness default	A cap that truncates reasoning scores the cap	32-token cap zeroed the reference model
2	Run the reference model under the same harness before trusting any comparison	A broken harness looks like a broken method	vendor 0.0000 at 32 tokens
3	Enforce or report a stop rule (`\nQuestion:`, EOS rate)	Continuation is scored as an arithmetic error	11–53% cross-question continuation
4	Report two extraction rules, or one anchored on the answer sentence	Last-number extraction reads numbers out of the continuation	4.1× → 1.3×
5	Archive full generations and token counts, not prefixes	No item-level audit is possible; re-scoring is impossible	200/300-character archives
6	Label the quantization dtype of every arm (per-channel vs group size)	Calibration effects confound with granularity	`W4A16` vs `W4A16_G64`

#	Requirement	Failure it prevents	Our case
7	Verify that the treatment artifact differs from the control (md5)	A silently ignored treatment yields a null result	three “different” builds, one md5
8	Report the group size with every ranking claim	The sign of the effect can flip	g64 −7.43 pp, g128 +6.71 pp

We do not claim this list is complete, but every entry is backed by a measured failure in this study rather than by intuition.

---

8 Limitations

1. **Single platform, single model family.** One SoC and one 1.5B model; the 3B model is a vendor-quantized reference without calibration variants. External validity is unproven.
2. **Board-level calibration variants are not yet validated.** The externally calibrated artifacts on the NPU were produced through a toolchain path whose parameter injection we have not yet demonstrated to take effect at 1.5B scale; moreover those artifacts were built at group size 64 (`W4A16_G64`) while the vendor reference and our local rebuilds are per-channel (`W4A16`), so the board-level “calibration” comparison confounds calibration with granularity. Board-side NAC results are therefore reported as pending, not as findings.
3. **Simulation is not deployment.** The x86 fp32 stack differs from the NPU in batch handling, prompt formatting, and numerical path; we use it only for paired relative comparisons and never for absolute prediction.
4. **Extraction sensitivity is bounded by archived prefixes.** Historical generations were archived truncated (300/200 characters), so re-scoring gives lower bounds only; the re-run archives full text.
5. **No energy measurement.** The board exposes voltage but not current; we report TTFT and decode throughput only.
6. **NAC-v2’s two changes are not separated in simulation.**

---

9 Conclusion

Under one W4A16 pipeline on one edge SoC, we find that calibration objectives that are indistinguishable on perplexity and multiple-choice accuracy differ by more than 4× on long-form reasoning; that this difference is invisible under a 32-token generation cap, which zeroes out even the vendor reference; and that the sign of the calibration effect depends on group size. The cap also changes the verdict outright: the same artifacts score 0.0150 or 0.5350, and 0.0050 or 0.5650, depending only on how many tokens the harness allows — and at the proper cap both are statistically indistinguishable from the vendor model. We release the harness, the archived generations, and a protocol that makes these distinctions reproducible. The broader lesson is that for edge LLM quantization, the evaluation protocol is part of the result.

Appendix A — Claims ledger

Claim	Evidence file	Status
GPTQ 0.305 / min-max 0.075 / NAC-v1 0.070 (sim GSM8K, 512 tok, 2-shot)	`acc_gsm8k_sim_g64.full.json` + records	verified
PPL 11.893 / 12.168 / 12.225 (64 seq)	`ppl_g64_64seq.json`	verified
AWQ 0.000 at n=64, answer-is 0.359	`acc_gsm8k_sim_g64_awq.json`, `extractor_recheck.json`	verified
MMLU p-values (NAC vs min-max, etc.)	`stats_pairs.json`	verified
g128 inversion (+6.71 pp, p=7.9e-9)	`acc_mmlu_sim_g128.full.json`, `stats_pairs.json`	verified
Vendor 0.0000 @32 vs 0.5550 @512	`acc_gsm8k_..._zs32.json`, `..._zs.json`	verified
Local rebuild 0.0150 @32 vs 0.5350 @512 (p = 0.69 vs vendor)	`acc_gsm8k_qwen1.5b_w4a16plain_x86_zs512.json`	verified
Group-quantized artifact 0.0050 @32 vs 0.5650 @512 (p = 0.88; answer-is identical)	`acc_gsm8k_q1bd8_zs512.json`	verified
Deployment MMLU build-path deltas and p-values	`acc_mmlu_*_zs.records.json`, `stats_pairs.json`	verified
fp16 GSM8K anchor	—	pending

Appendix B — Reproducibility package

- `src/eval/accuracy.py`: deployment harness (full-generation archive, token counts, dual extraction).
- `scripts/eval_gsm8k_sim.py`: simulation harness with the same protocol.
- `scripts/q1_board512_queue.sh`: the 512-token re-measurement queue.
- `results/env_snapshot.json`: software versions and artifact md5s.
- `reproduce/MANIFEST.md`: every artifact with md5, dtype and role.
- `reproduce/README.md`: per-table commands and an explicit list of gaps.
- `analysis/`: independent re-scoring scripts used for the audit in this paper.

Appendix C — Pairwise comparisons

McNemar's test with continuity correction on paired items; `OR = b/c` where `b` counts items the first configuration gets right and the second gets wrong. Differences and CIs are in percentage points; positive means the second configuration is better.

Table 5. Pairwise comparisons.

Comparison	n	acc(A)	acc(B)	Δ (pp)	OR	95% CI (pp)	p
MMLU g64: NAC-v2 vs NAC-v1	969	0.393	0.447	+5.37	2.13	[+3.0, +7.7]	2.1e-5
MMLU g64: NAC-v2 vs GPTQ	969	0.452	0.447	−0.52	0.92	[−2.8, +1.7]	0.72
MMLU g64: NAC-v2 vs min-max	969	0.468	0.447	−2.06	0.75	[−4.4, +0.3]	0.11
MMLU g64: NAC-v2 vs fp16	969	0.463	0.447	−1.65	0.72	[−3.7, +0.4]	0.13
MMLU g64: AWQ vs GPTQ	969	0.452	0.440	−1.24	0.85	[−3.8, +1.2]	0.37
MMLU g64: AWQ vs NAC-v2	969	0.447	0.440	−0.72	0.89	[−2.9, +1.4]	0.58
MMLU g64: AWQ vs min-max	969	0.468	0.440	−2.79	0.69	[−5.3, −0.4]	0.033
MMLU g64: AWQ vs fp16	969	0.463	0.440	−2.37	0.66	[−4.5, −0.2]	0.038
MMLU g64: NAC-v1 vs min-max	969	0.468	0.393	−7.43	0.33	[−9.9, −5.2]	3.3e-9
GSM8K: GPTQ vs min-max	200	0.075	0.305	+23.00	7.57	[+16.0, +30.0]	6.3e-9
GSM8K: GPTQ vs NAC-v1	200	0.070	0.305	+23.50	7.71	[+16.5, +30.5]	3.9e-9
GSM8K: NAC-v2 vs NAC-v1	200	0.070	0.150	+8.00	2.60	[+2.5, +14.0]	0.012
GSM8K: NAC-v2 vs GPTQ	200	0.305	0.150	−15.50	0.33	[−23.0, −8.0]	1.2e-4
GSM8K: NAC-v2 vs min-max	200	0.075	0.150	+7.50	2.25	[+1.5, +13.5]	0.025
GSM8K: AWQ vs GPTQ	64	0.312	0.000	−31.25	0.00	[−42.2, −20.3]	2.2e-5
GSM8K: AWQ vs NAC-v2	64	0.141	0.000	−14.06	0.00	[−23.4, −6.2]	0.0077
GSM8K: AWQ vs min-max	64	0.047	0.000	−4.69	0.00	[−10.9, +0.0]	0.25
MMLU g128: NAC-v1 vs min-max	969	0.362	0.429	+6.71	3.24	[+4.5, +8.9]	7.9e-9
MMLU g128: GPTQ vs min-max	969	0.362	0.487	+12.49	3.16	[+9.5, +15.5]	3.8e-15
MMLU g128: GPTQ vs NAC-v1	969	0.429	0.487	+5.78	1.86	[+3.0, +8.6]	5.5e-5

The AWQ rows use n = 64 and their odds ratios are zero by construction (no correct items), so only the CI is informative. Two rows are non-significant and are reported as

ties rather than as trends: NAC-v2 vs GPTQ on MMLU ($p = 0.72$) and NAC-v2 vs min-max ($p = 0.11$).

References

1. E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. ICLR 2023.
2. J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, S. Han. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. MLSys 2024.
3. G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, S. Han. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. ICML 2023.
4. W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, P. Luo. OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models. ICLR 2024.
5. S. Dutta et al. Accuracy Is Not All You Need. NeurIPS 2024. arXiv:2407.09141.
6. A Case Study of Selected PTQ Baselines for Reasoning LLMs on Ascend NPU. arXiv: 2602.17693, 2026.
7. J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, D. Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.
8. K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, J. Schulman. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168, 2021.
9. D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, J. Steinhardt. Measuring Massive Multitask Language Understanding. ICLR 2021.
10. Qwen Team. Qwen2.5 Technical Report. arXiv:2412.15115, 2024.
11. T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. NeurIPS 2022.
12. Z. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, Y. He. ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers. NeurIPS 2022.
13. T. Dettmers, L. Zettlemoyer. The case for 4-bit precision: k-bit Inference Scaling Laws. ICML 2023.
14. S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, M. Jaggi, D. Alistarh, T. Hoefler, J. Hensman. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. NeurIPS 2024.
15. S. Merity, C. Xiong, J. Bradbury, R. Socher. Pointer Sentinel Mixture Models. ICLR 2017. (WikiText-2)